# Project 3: Modelling the COVID-19 Shock in South African Accommodation Demand

**Section 2: Exponential smoothing, time series regression and state-space models**

Eden

## Table of contents

**Executive summary** **2**

**1 Introduction and research question** **2**

**2 Data** **3**
2.1 Source and data statement . . . 3
2.2 Reading the data . . . 3
2.3 Training and test sets . . . 6

**3 Model 1: Exponential smoothing (ETS)** **6**

**4 Model 2: Regression with ARIMA errors and intervention terms** **7**
4.1 Why deterministic seasonality . . . 7
4.2 Modelling the shock . . . 7
4.3 Choosing the decay rate by profile likelihood . . . 8
4.4 Fitting the model (Box and Jenkins loop) . . . 9
4.5 Residual diagnostics . . . 11
4.6 Does the exchange rate help? . . . 12

**5 Model 3: Basic structural model with the Kalman filter** **13**

**6 Forecast evaluation** **16**
6.1 Forecasts on the test set . . . 16
6.2 Results . . . 20

**7 Comparison with Project 1** **21**

**8 Discussion** **22**
8.1 Key findings . . . 22
8.2 Issues and limitations . . . 22
8.3 What I would do differently . . . 23

**Reproducibility** **23**

## Executive summary


**Executive summary**

**Title:** Modelling the COVID-19 shock in South African accommodation demand
**Section and topic:** Section 2. Exponential smoothing (ETS), regression with ARIMA errors and intervention (transfer function) terms, state-space models and the Kalman filter.
**Type of project:** Model comparison on real-world data, with a profile likelihood search for the intervention decay rate. It extends Project 1 on the same series and the same test set, so the results of the two projects can be compared directly.
**Question:** In Project 1 the COVID-19 months were either kept, which broke the SARIMA model, or deleted, which threw information away. Can the shock be *modelled* instead, and does that, or adding the exchange rate as a predictor, give better forecasts of accommodation demand than ETS and the Project 1 SARIMA?
**Data:** Real-world, publicly available data. Statistics South Africa, *Tourist Accommodation* release P6410, series B64100 (stay unit nights sold, total industry), and the monthly average rand per US dollar exchange rate (FRED series EXSFUS), January 2015 to June 2026. Full source details are in Section 2.1.
**Methods:** ETS; regression with ARIMA errors containing deterministic seasonal dummies and COVID intervention terms (a geometrically decaying shock, two pulses and a level shift), with the decay rate chosen by profile likelihood; the same model with the log exchange rate added, forecast both ex ante and ex post; a basic structural model (local level, slope and seasonal) fitted by maximum likelihood with the Kalman filter, with the COVID months treated as missing. All models go through the Box and Jenkins loop and are compared with the Project 1 SARIMA and the seasonal naive benchmark on the July 2024 to June 2026 test set.
**Key findings:** See Section 8.1.
**What I would do differently:** Use rolling-origin cross-validation for the final comparison, and estimate the decay rate jointly with the other parameters rather than on a grid.
**Detailed analysis:** the sections below.


## 1 Introduction and research question

Project 1 showed that the COVID-19 collapse in April 2020 is the main obstacle to forecasting South African accommodation demand. Keeping the COVID months in a SARIMA model inflated the residual variance many times over and pushed the seasonal MA coefficient to the invertibility boundary. Treating the months as missing solved this, but at the cost of discarding two years of data. This project asks whether the shock can be modelled directly, using the Section 2 tools:

- **ETS** as a baseline: exponential smoothing adapts its level quickly, which might let it recover from the shock without any special treatment.
- **Regression with ARIMA errors and intervention terms:** the shock enters as explanatory variables, so the months are used rather than deleted, and the size and persistence of the shock are estimated.
- **A basic structural model (BSM):** a state-space model with an evolving level, slope and seasonal pattern. The Kalman filter handles missing observations naturally, so the COVID months can be skipped without breaking the time ordering, and the Kalman smoother estimates the underlying level through the gap.

A second question is whether the exchange rate helps. A weaker rand makes South Africa cheaper for foreign visitors, so the rand might carry information about demand. This links to Project 2, which models the rand's volatility.

**Research question.** *Does modelling the COVID-19 shock explicitly (with intervention terms or as missing data in a state-space model), or adding the exchange rate, forecast monthly accommodation demand better than ETS and the Project 1 SARIMA?*

# 2 Data

## 2.1 Source and data statement

**Data sources at a glance**

| | |
|---|---|
| **Accommodation data** | Statistics South Africa (Stats SA), *Tourist Accommodation*, statistical release P6410 (monthly), file `tourist_data.xlsx`, series B64100 (stay units nights sold, total industry, actual figures, thousands). Available from https://www.statssa.gov.za/?page_id=1854&PPN=P6410 |
| **Exchange rate** | Board of Governors of the Federal Reserve System (US), *South African Rand to U.S. Dollar Spot Exchange Rate* (EXSFUS), monthly average of daily rates, rand per US dollar, via FRED: https://fred.stlouisfed.org/series/EXSFUS. Saved as `usdzar_monthly_fred.csv` |
| **Period analysed** | January 2015 to June 2026 |
| **Licence and privacy** | Public, aggregated official statistics; no personal information, so POPIA does not apply |

**References:**


- Statistics South Africa (2026) *Tourist Accommodation (Statistical release P6410)*, time series data. Pretoria: Statistics South Africa. Available at: https://www.statssa.gov.za/?page_id=1854&PPN=P6410.
- Board of Governors of the Federal Reserve System (US) (2026) *South African Rand to U.S. Dollar Spot Exchange Rate [EXSFUS]*. Retrieved from FRED, Federal Reserve Bank of St. Louis. Available at: https://fred.stlouisfed.org/series/EXSFUS.


## 2.2 Reading the data

The accommodation series is read exactly as in Project 1. The exchange rate is downloaded from FRED the first time the document is rendered and saved as `usdzar_monthly_fred.csv`; later renders use the

saved file. If the download fails, download the CSV from the FRED page above and save it under that name in the same folder as this file.

```
# Accommodation (as in Project 1) ----------------------------------------
raw <- read_excel(tourism_file)

nights_sold <- raw |>
  filter(H03 == "B64100") |>
  select(starts_with("MO")) |>
  pivot_longer(everything(), names_to = "code", values_to = "nights") |>
  mutate(Month  = yearmonth(make_date(as.integer(substr(code, 5, 8)),
                                      as.integer(substr(code, 3, 4)), 1L)),
         nights = as.numeric(nights)) |>
  filter(!is.na(nights)) |>
  select(Month, nights) |>
  as_tsibble(index = Month) |>
  filter_index("2015 Jan" ~ .) |>
  mutate(period = case_when(
    Month <  covid_start ~ "Pre-COVID",
    Month <= covid_end   ~ "COVID window",
    TRUE                 ~ "Post-COVID"
  ))

# Exchange rate ----------------------------------------------------------
if (!file.exists(zar_file)) {
  fred_url <- paste0("https://fred.stlouisfed.org/graph/fredgraph.csv?id=EXSFUS",
                     "&cosd=2015-01-01&coed=2026-06-01")
  download.file(fred_url, zar_file, mode = "wb")
}

zar <- read.csv(zar_file, na.strings = c(".", ""))
names(zar) <- c("date", "zar")
zar <- zar |>
  mutate(Month = yearmonth(as.Date(date)), zar = as.numeric(zar)) |>
  filter(!is.na(zar)) |>
  select(Month, zar)

nights_sold <- nights_sold |>
  left_join(zar, by = "Month") |>
  mutate(log_zar = log(zar))

sum(is.na(nights_sold$zar))   # should be 0
```

```
[1] 0
```

```
p_n <- nights_sold |>
  ggplot(aes(as.Date(Month), nights)) +
  annotate("rect", xmin = as.Date(covid_start), xmax = as.Date(covid_end),
           ymin = -Inf, ymax = Inf, alpha = 0.15, fill = "firebrick") +
  geom_line(colour = "steelblue4") +
```

```
  scale_y_continuous(labels = scales::label_comma()) +
  labs(x = NULL, y = "Nights sold (thousands)")

p_z <- nights_sold |>
  ggplot(aes(as.Date(Month), zar)) +
  annotate("rect", xmin = as.Date(covid_start), xmax = as.Date(covid_end),
           ymin = -Inf, ymax = Inf, alpha = 0.15, fill = "firebrick") +
  geom_line(colour = "darkorange3") +
  labs(x = NULL, y = "ZAR per USD")

if (requireNamespace("patchwork", quietly = TRUE)) {
  patchwork::wrap_plots(p_n, p_z, ncol = 1)
} else {
  print(p_n); print(p_z)
}
```

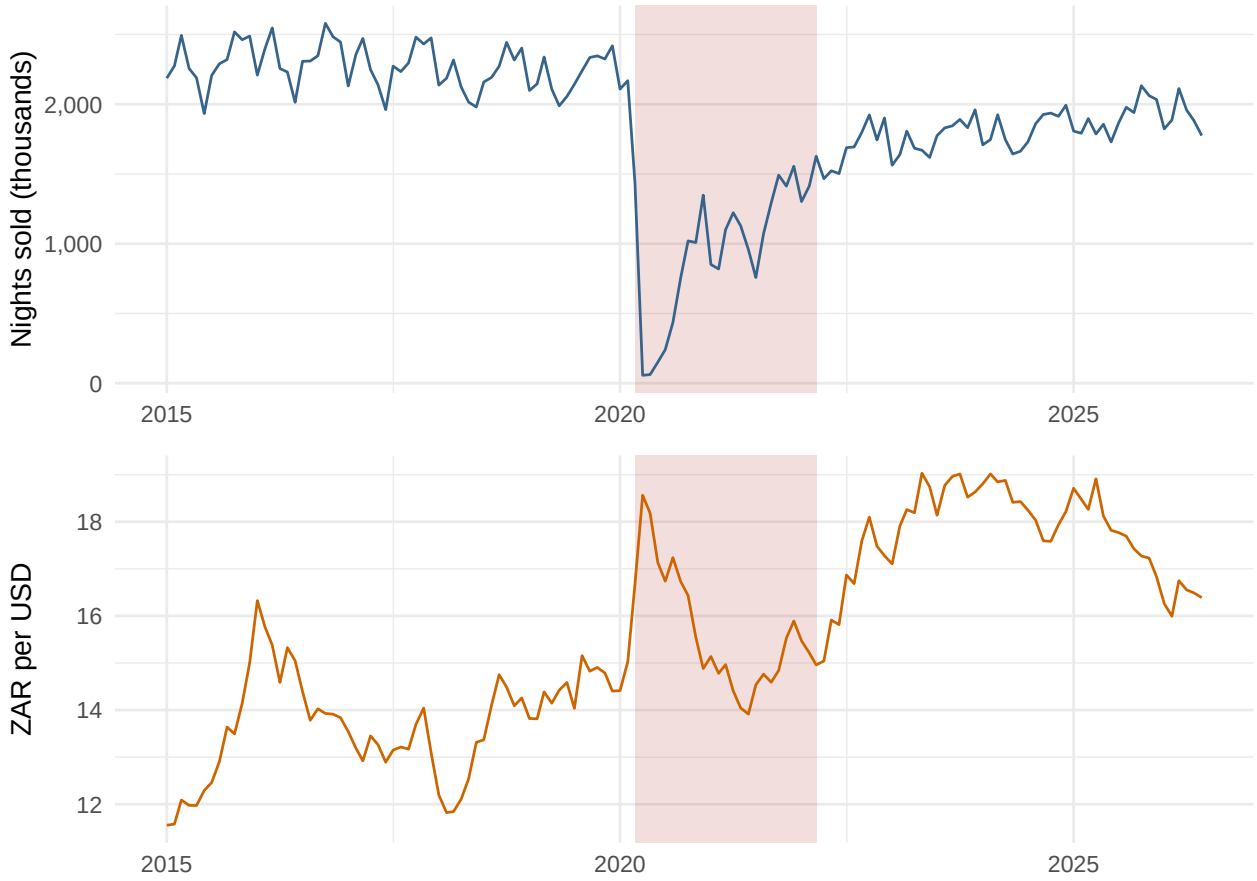



Figure 1: Top: monthly stay unit nights sold (thousands), with the COVID window shaded. Bottom: monthly average rand per US dollar.

The full exploratory analysis of the accommodation series is in Project 1 and is not repeated here. The points that matter for this project are: the seasonal shape is very stable from year to year; the April 2020 collapse was followed by a recovery that took about two years, with setbacks in the later COVID

waves; and the post-COVID level is still slightly below the pre-COVID level. The rand weakened over the period, with a sharp depreciation at the start of the pandemic (Figure 1), so the exchange rate and the COVID shock are partly confounded in 2020. This is one reason the exchange rate is added only *after* the shock has been modelled.

### 2.3 Training and test sets

```
train   <- nights_sold |> filter(Month < test_start)
test    <- nights_sold |> filter(Month >= test_start)
train_B <- train |>
  mutate(nights = if_else(period == "COVID window", NA_real_, nights))
```

The training and test sets are identical to Project 1: training from January 2015 to June 2024 (114 months) and testing from July 2024 to June 2026 (24 months). Using the same split is what allows the results to be compared across the two projects.

## 3 Model 1: Exponential smoothing (ETS)

ETS models describe a series through its level, trend and seasonal components, each updated by exponential smoothing. The `ETS()` function in `fable` selects the error, trend and seasonal forms (additive, multiplicative or none) by AICc. ETS cannot handle missing values, so it is fitted to the full training series *including* the COVID months. This makes ETS a useful baseline: it shows how far a method that adapts its level quickly can cope with the shock on its own.

```
fit_ets <- train |> model(ETS = ETS(nights))
report(fit_ets)
```

```
Series: nights
Model: ETS(A,N,A)
  Smoothing parameters:
    alpha = 0.9998983
    gamma = 0.000100251

  Initial states:
     l[0]     s[0]    s[-1]    s[-2]    s[-3]     s[-4]     s[-5]     s[-6]
 2326.369 272.1421 157.9485 228.8554 64.78211 -46.92921 -135.0183 -283.3306
     s[-7]     s[-8]    s[-9]  s[-10]    s[-11]
 -224.3569 -164.0777 124.3326 53.8321 -48.18011

  sigma^2:  30302.51

     AIC     AICc      BIC
1731.354 1736.252 1772.397
```

```
ets_spec <- fabletools::model_sum(fit_ets$ETS[[1]]$fit)
ets_par  <- tidy(fit_ets)
alpha_ets <- ets_par$estimate[ets_par$term == "alpha"]
```

The selected model is ETS(A,N,A). The smoothing parameter for the level is $\hat{\alpha} = 1.000$. A value this large means the level is updated almost entirely from the latest observation, which is how ETS follows the collapse and the recovery: it simply resets its level each month. The cost is that the level forecast at the end of the training period rests heavily on the last few observations.

# 4 Model 2: Regression with ARIMA errors and intervention terms

## 4.1 Why deterministic seasonality

In Project 1, every seasonal ARIMA model fitted with the COVID months included estimated a seasonal MA coefficient at or near -1. A seasonal MA factor $(1 - L^{12})$ that cancels the seasonal difference $(1 - L^{12})$ is the classic sign that the seasonal pattern is fixed rather than evolving, so that seasonal differencing is over-differencing. The seasonal plots in Project 1 point the same way. In this model the seasonality is therefore represented by 11 monthly dummy variables, while a first difference in the errors handles the wandering level.

## 4.2 Modelling the shock

The intervention terms follow the transfer function idea from Chapter 2: the effect of an event is described by a small set of explanatory variables whose shape reflects how the event unfolded. With $t_0$ = April 2020 and everything on the log scale:

- **Decaying shock:** $D_t(\delta) = \delta^{\,t-t_0}$ for $t \geq t_0$ and 0 before. Its coefficient $\omega$ is the immediate effect in April 2020, which then dies out geometrically at rate $\delta$. This is the transfer function $\omega/(1-\delta L)$ applied to a pulse at $t_0$.
- **Pulses** for March 2020 (lockdown started on 27 March, so the month was only partly affected) and May 2020 (the industry stayed almost completely closed, so the bottom of the collapse lasted two months rather than one).
- **Level shift:** $S_t = 1$ for $t \geq t_0$, allowing a permanent difference between the pre-COVID and post-COVID levels.

The model is

$$\log y_t = \sum_{j=1}^{11} \gamma_j M_{jt} + \omega D_t(\delta) + \kappa_1 P_t^{\text{Mar20}} + \kappa_2 P_t^{\text{May20}} + \lambda S_t + \eta_t, \qquad (1 - L)\,\phi(L)\,\eta_t = \theta(L)\,\varepsilon_t,$$

where $M_{jt}$ are monthly dummies and $\eta_t$ follows an ARIMA($p$, 1, $q$) process.

```
covid_vars <- function(df, delta) {
  k0 <- month_num(shock_month)
  df |>
    mutate(k     = month_num(Month) - k0,
           decay = if_else(k >= 0, delta^pmax(k, 0), 0),
           mar20 = as.numeric(k == -1),
           may20 = as.numeric(k == 1),
           step  = as.numeric(k >= 0))
}
```

### 4.3 Choosing the decay rate by profile likelihood

The decay rate $\delta$ enters the model non-linearly, so it cannot be estimated by `ARIMA()` directly. Instead I profile it: for each $\delta$ on a grid from 0.70 to 0.96, the model is fitted (with the ARIMA error orders chosen by AICc) and the AICc recorded. The value of $\delta$ with the smallest AICc is used. Because $\delta$ is estimated, the model really has one more parameter than `ARIMA()` reports; this does not change the ranking along the grid.

```
delta_grid <- seq(0.70, 0.96, by = 0.02)

profile <- bind_rows(lapply(delta_grid, function(dl) {
  f <- covid_vars(train, dl) |>
    model(reg = ARIMA(log(nights) ~ season() + decay + mar20 + may20 + step +
                        pdq(d = 1) + PDQ(0, 0, 0)))
  tibble(delta = dl,
         AICc  = glance(f)$AICc,
         spec  = fabletools::model_sum(f$reg[[1]]$fit))
}))

delta_hat <- profile$delta[which.min(profile$AICc)]

ggplot(profile, aes(delta, AICc)) +
  geom_line(colour = "grey40") +
  geom_point(colour = "grey40") +
  geom_point(data = filter(profile, delta == delta_hat), colour = "firebrick", size =
   ↪  3) +
  labs(x = expression(delta), y = "AICc")
```

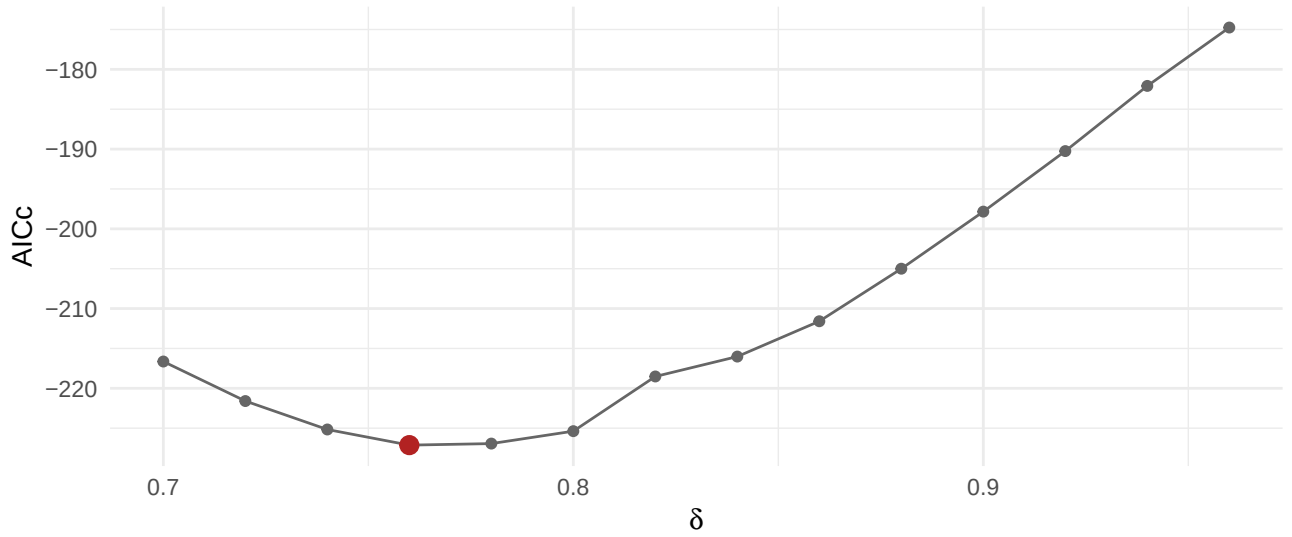



Figure 2: Profile of AICc against the decay rate. The selected value is marked in red.

The AICc is minimised at $\hat{\delta} = 0.76$ (Figure 2). A decay rate of 0.76 means that the temporary part of the shock halves every 2.5 months and has fallen to 4% of its initial size after a year.

### 4.4 Fitting the model (Box and Jenkins loop)

```
train_reg <- covid_vars(train, delta_hat)
test_reg  <- covid_vars(test, delta_hat)

# Step 2 and 3: residuals from the regression part, before choosing the ARIMA errors
ols <- train_reg |>
  model(TSLM(log(nights) ~ season() + decay + mar20 + may20 + step))
ols_resid <- augment(ols)

ols_resid |>
  gg_tsdisplay(difference(.resid), plot_type = "partial", lag_max = 36)
```

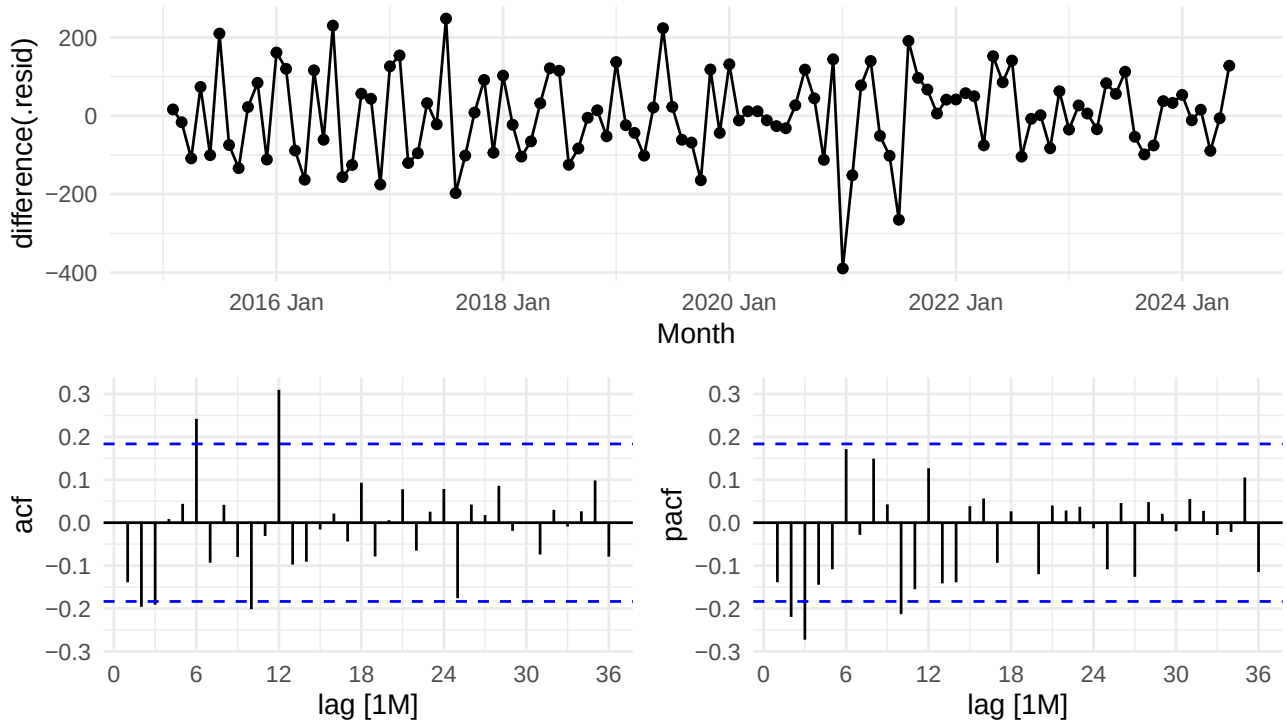



Figure 3: Box and Jenkins steps 2 and 3 for the error process. First differences of the residuals from the regression part, with their ACF and PACF.

Following the Box and Jenkins workflow, the regression residuals are examined before choosing the error model. The residuals from the regression part wander (they are a non-stationary level once the shock is removed), so a first difference is taken. Figure 3 shows the differenced residuals with their ACF and PACF, which suggest a low order ARMA structure for the error. The orders are then chosen by AICc, and the model is estimated by maximum likelihood.

```
fit_reg <- train_reg |>
  model(
    Reg = ARIMA(log(nights) ~ season() + decay + mar20 + may20 + step +
                  pdq(d = 1) + PDQ(0, 0, 0)),
    `Reg + ZAR` = ARIMA(log(nights) ~ season() + decay + mar20 + may20 + step +
                          log_zar + pdq(d = 1) + PDQ(0, 0, 0))
```

```
  )

reg_spec <- fabletools::model_sum(fit_reg$Reg[[1]]$fit)
zar_spec <- fabletools::model_sum(fit_reg$`Reg + ZAR`[[1]]$fit)

reg_coef <- tidy(fit_reg) |>
  filter(term %in% c("decay", "mar20", "may20", "step", "log_zar"))

reg_coef |>
  select(Model = .model, Term = term, Estimate = estimate,
         `Std. error` = std.error, `p-value` = p.value) |>
  kable(digits = 3)
rc <- function(m, t, col = "estimate") reg_coef[[col]][reg_coef$.model == m &
 ↪  reg_coef$term == t]
```

Table 2: Estimated intervention and exchange rate coefficients (log scale). Seasonal dummy and ARMA coefficients are omitted for space.

| Model | Term | Estimate | Std. error | p-value |
|---|---|---|---|---|
| Reg | decay | -3.187 | 0.133 | 0.000 |
| Reg | mar20 | -0.542 | 0.073 | 0.000 |
| Reg | may20 | -0.531 | 0.057 | 0.000 |
| Reg | step | -0.515 | 0.112 | 0.000 |
| Reg + ZAR | decay | -3.262 | 0.135 | 0.000 |
| Reg + ZAR | mar20 | -0.576 | 0.076 | 0.000 |
| Reg + ZAR | may20 | -0.541 | 0.058 | 0.000 |
| Reg + ZAR | step | -0.517 | 0.099 | 0.000 |
| Reg + ZAR | log_zar | 0.270 | 0.198 | 0.175 |

```
effect <- train_reg |>
  as_tibble() |>
  mutate(log_effect = rc("Reg", "decay") * decay + rc("Reg", "mar20") * mar20 +
           rc("Reg", "may20") * may20 + rc("Reg", "step") * step,
         lost = 1 - exp(log_effect)) |>
  filter(Month >= yearmonth("2019 Jul"))

ggplot(effect, aes(as.Date(Month), 100 * lost)) +
  geom_col(fill = "firebrick", alpha = 0.7) +
  labs(x = NULL, y = "Demand lost to COVID (%)")
```

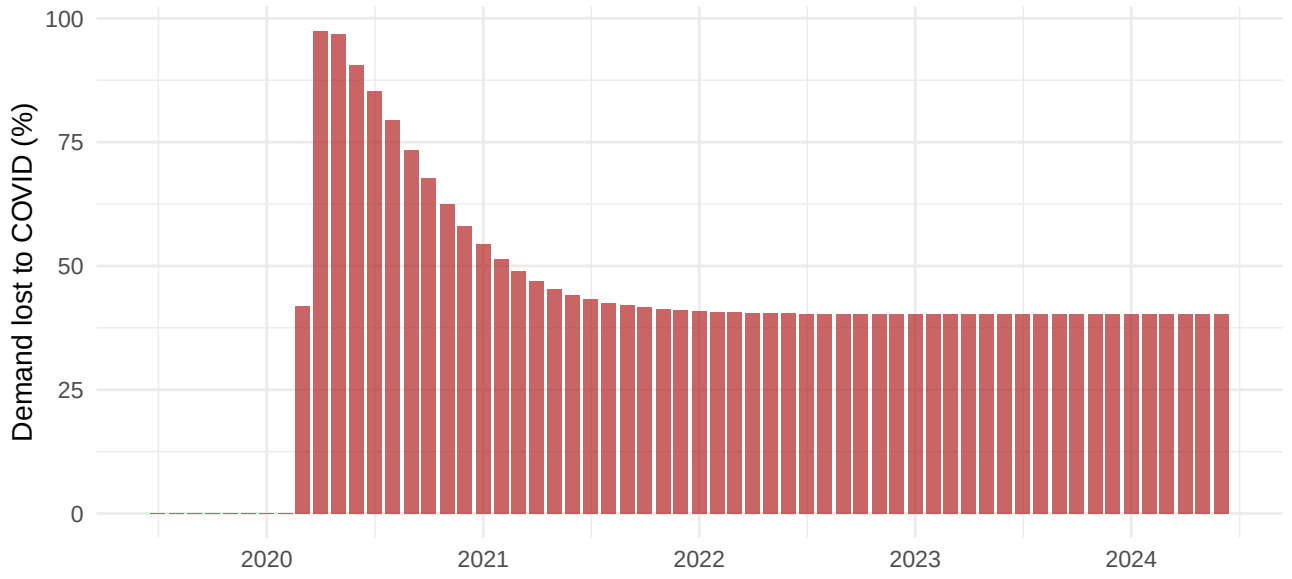



Figure 4: Estimated total COVID effect on log nights sold from the regression model (decay, pulses and level shift combined), expressed as the percentage of demand lost relative to a no-COVID baseline.

The selected error model is LM w/ ARIMA(2,1,1) errors. The estimates have a direct interpretation on the log scale. The decaying shock has $\hat{\omega}$ = -3.19; combined with the May 2020 pulse and the level shift, this reproduces a loss of almost all demand in April and May 2020. The level shift is $\hat{\lambda}$ = -0.515 ($p < 0.001$), which corresponds to a lasting change of -40.3% in the level of demand compared with what the pre-COVID pattern would imply. Figure 4 turns these estimates into the estimated share of demand lost each month.

### 4.5 Residual diagnostics

```
reg_aug <- augment(fit_reg)

n_arma <- tidy(fit_reg) |>
  filter(grepl("^(ar|ma)[0-9]+$", term)) |>
  count(.model, name = "dof")

reg_lb <- reg_aug |>
  as_tibble() |>
  left_join(n_arma, by = ".model") |>
  mutate(dof = coalesce(dof, 0L)) |>
  group_by(.model) |>
  summarise(lb_p = Box.test(.innov, lag = 24, type = "Ljung-Box",
                            fitdf = first(dof))$p.value,
            .groups = "drop")

lbp <- function(m) reg_lb$lb_p[reg_lb$.model == m]

fit_reg |> select(Reg) |> gg_tsresiduals(lag_max = 36)
```

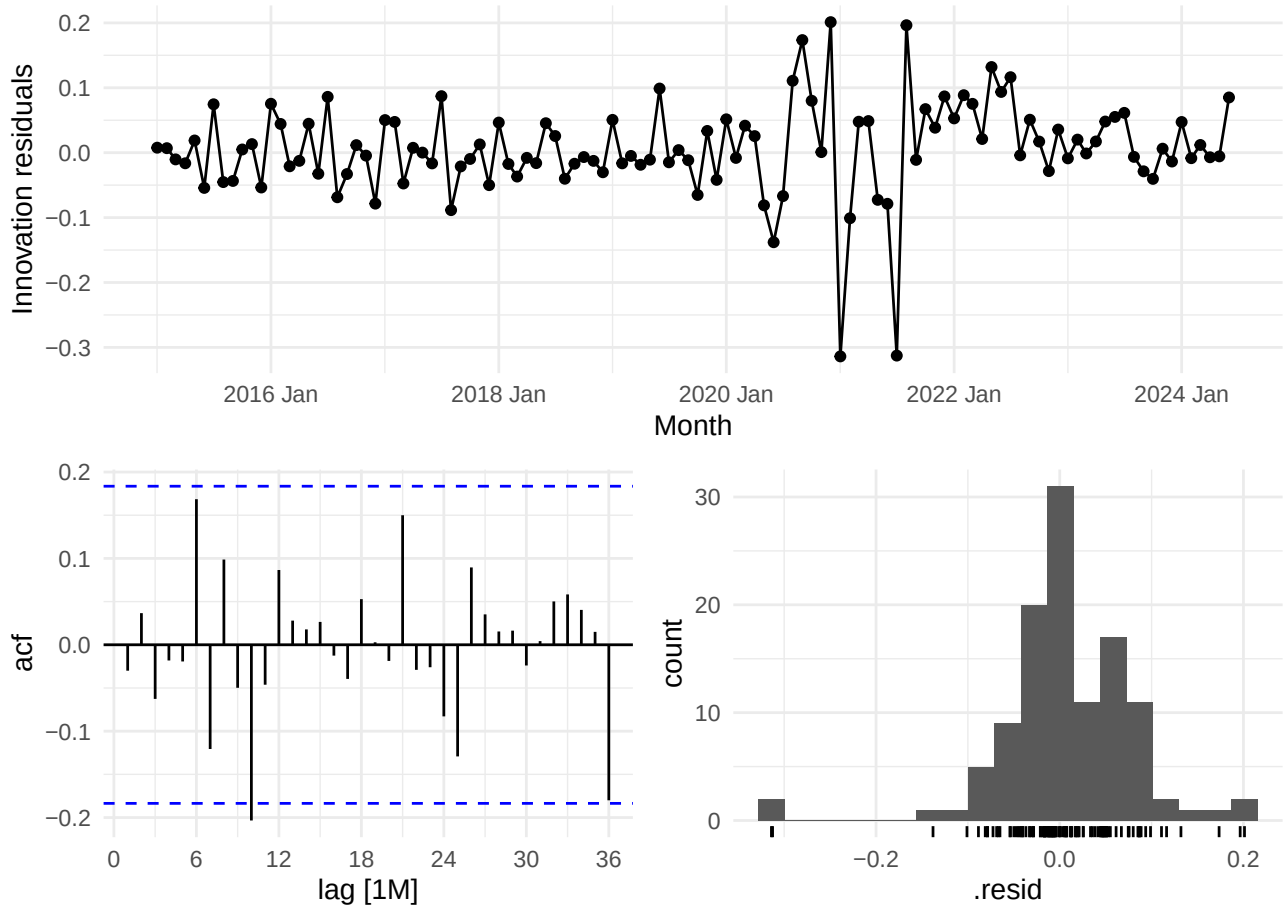



Figure 5: Residual diagnostics for the regression model with ARIMA errors (without the exchange rate).

The Ljung and Box test on 24 lags gives p = 0.556 for the regression model and p = 0.544 for the model with the exchange rate. The residuals of the regression model are consistent with white noise and show no remaining seasonal structure, so the deterministic seasonal dummies are adequate. Unlike the Project 1 SARIMA fitted to the full data, the COVID months no longer dominate the residuals, because the shock is now explained by the intervention terms. The model passes step 5 and can be used to forecast.

### 4.6 Does the exchange rate help?

Because the model is in logs, the coefficient on log USD/ZAR is an elasticity: a 10% weakening of the rand is associated with a change of about 2.7% in nights sold, holding the seasonal and COVID terms fixed. The estimate is 0.270 (p 0.175), and the AICc gets worse from -227.1 to -225.9. The exchange rate does not add significant information once the seasonal pattern and the COVID shock are accounted for, which is not surprising since most stays are domestic and the effect of the rand on foreign arrivals works with a lag. The model is in first differences, so the coefficient measures the effect of *changes* in the rand on *changes* in demand within the same month.

A model with an explanatory variable has a practical catch: to forecast nights sold, I need future values of the exchange rate, which are not known when the forecast is made. Two versions are therefore evaluated:

- **Ex ante (realistic):** the exchange rate is assumed to stay at its last observed value (June 2024), a naive forecast.
- **Ex post (optimistic):** the actual exchange rate during the test period is used. This is not a genuine forecast, since it uses information that would not have been available in June 2024. It shows the best the model could do if the rand were forecast perfectly.

## 5 Model 3: Basic structural model with the Kalman filter

The basic structural model (BSM) writes the log series as the sum of unobserved components, each evolving as a random walk:

$$\begin{aligned} \log y_t &= \mu_t + s_t + \varepsilon_t, & \varepsilon_t &\sim N(0, \sigma_\varepsilon^2) \\ \mu_t &= \mu_{t-1} + \nu_{t-1} + \xi_t, & \xi_t &\sim N(0, \sigma_\xi^2) \\ \nu_t &= \nu_{t-1} + \zeta_t, & \zeta_t &\sim N(0, \sigma_\zeta^2) \\ s_t &= -\textstyle\sum_{j=1}^{11} s_{t-j} + \omega_t, & \omega_t &\sim N(0, \sigma_\omega^2) \end{aligned}$$

The first line is the observation equation; the others are the state equations for the level $\mu_t$, the slope $\nu_t$ and the seasonal effect $s_t$. The four variances are estimated by maximum likelihood, where the likelihood comes from the prediction error decomposition computed by the Kalman filter. The same model can be built in `dlm` as `dlmModPoly(2) + dlmModSeas(12)`; here I use the base R function `StructTS()`, which fits exactly this model.

The COVID months are set to missing (as in Design B of Project 1). At a missing observation the Kalman filter performs only the prediction step, so the states are carried through the gap and the likelihood is computed from the observed months only.

Two versions are fitted: the full BSM, and a version with the slope variance fixed at zero (a fixed drift), which is a restriction suggested by the size of the estimated slope variance.

```
y_B <- ts(log(train_B$nights), start = c(2015, 1), frequency = 12)

bsm_free  <- StructTS(y_B, type = "BSM")
bsm_fixed <- StructTS(y_B, type = "BSM", fixed = c(NA, 0, NA, NA))

tibble(
  Model = c("BSM", "BSM (fixed slope)"),
  `Level var` = c(bsm_free$coef["level"], bsm_fixed$coef["level"]),
  `Slope var` = c(bsm_free$coef["slope"], bsm_fixed$coef["slope"]),
  `Seasonal var` = c(bsm_free$coef["seas"], bsm_fixed$coef["seas"]),
  `Irregular var` = c(bsm_free$coef["epsilon"], bsm_fixed$coef["epsilon"]),
  `Log likelihood` = c(bsm_free$loglik, bsm_fixed$loglik)
) |>
  kable(digits = c(0, 6, 7, 6, 6, 2))
```

Table 3: Estimated variances of the basic structural models (log scale) and log likelihoods.

| Model | Level var | Slope var | Seasonal var | Irregular var | Log likelihood |
|---|---|---|---|---|---|
| BSM | 0.001089 | 5e-07 | 0.000255 | 0 | 181.23 |
| BSM (fixed slope) | 0.001460 | 0e+00 | 0.000188 | 0 | 181.31 |

The estimated slope variance in the full BSM is tiny (5.2e-07), and fixing it at zero changes the log likelihood by only 0.08. The restricted model even has a slightly higher log likelihood, which should be impossible for nested models fitted by maximum likelihood: the optimiser for the full model stopped slightly short of the maximum. Variances close to zero sit on the boundary of the parameter space, where the likelihood is very flat and numerical optimisation is unreliable, which is a known difficulty with structural models. The seasonal variance is small compared with the level variance, which agrees with the finding that the seasonal pattern is very stable. As the forecasts below show, the tiny slope variance nevertheless matters for forecasting: it lets the slope adapt to the steep post-COVID recovery, and that recovery growth is then extrapolated into the test period.

```
sm <- tsSmooth(bsm_fixed)
smooth_df <- tibble(Month = train_B$Month,
                    obs = log(train_B$nights),
                    level = as.numeric(sm[, "level"]))

ggplot(smooth_df, aes(as.Date(Month))) +
  annotate("rect", xmin = as.Date(covid_start), xmax = as.Date(covid_end),
           ymin = -Inf, ymax = Inf, alpha = 0.15, fill = "firebrick") +
  geom_line(aes(y = obs), colour = "black", linewidth = 0.4) +
  geom_line(aes(y = level), colour = "firebrick", linewidth = 0.8) +
  labs(x = NULL, y = "log nights sold")
```

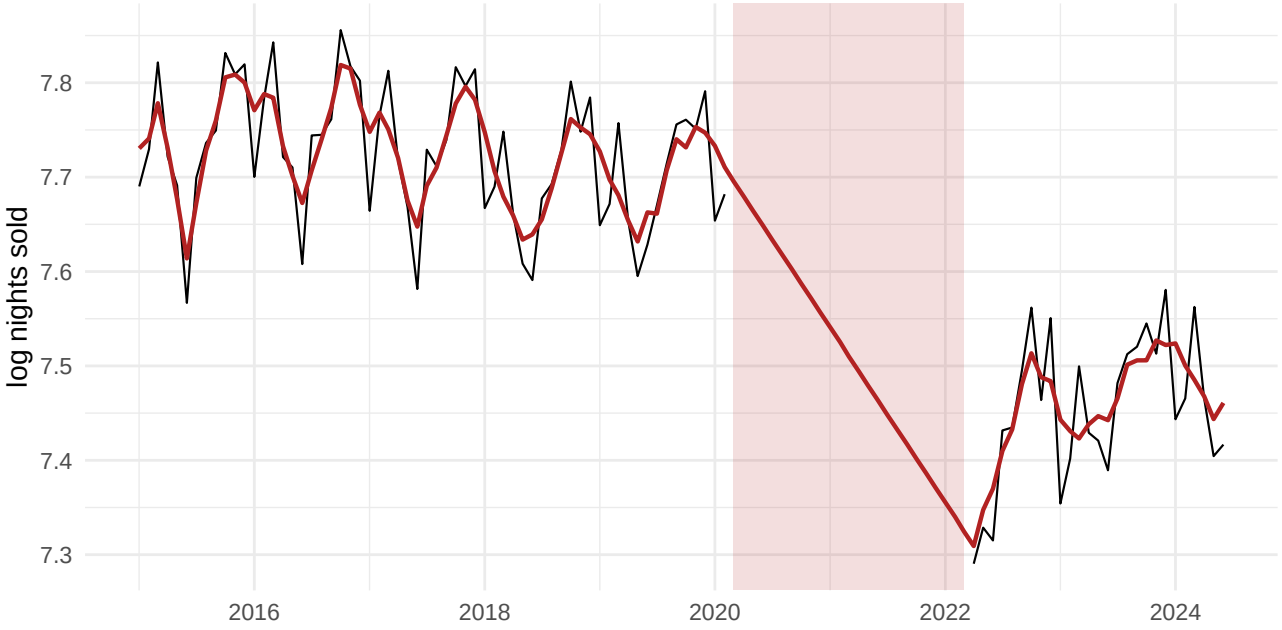



Figure 6: Log nights sold (black, with the COVID months removed) and the Kalman smoother estimate of the level of the fixed slope BSM (red). Inside the shaded COVID window there are no observations, and the smoothed level is an interpolation between the pre-COVID and post-COVID data.

```
acf_b <- acf(res_fixed, lag.max = 36, na.action = na.pass, plot = FALSE)
tibble(lag = acf_b$lag[-1], acf = acf_b$acf[-1]) |>
  ggplot(aes(lag, acf)) +
  geom_hline(yintercept = 0) +
  geom_hline(yintercept = c(-1, 1) * 1.96 / sqrt(sum(!is.na(res_fixed))),
             linetype = "dashed", colour = "steelblue") +
  geom_segment(aes(xend = lag, yend = 0)) +
  labs(x = "Lag (months)", y = "ACF")
```

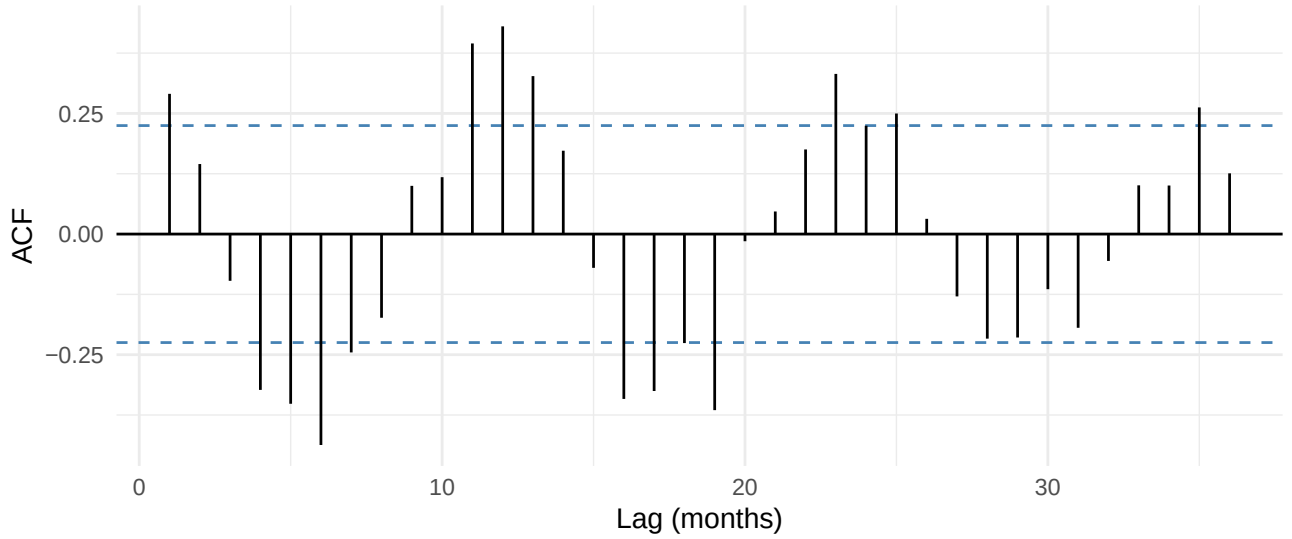



Figure 7: ACF of the standardised one-step prediction residuals of the fixed slope BSM (missing months passed through).

**Diagnostics.** The standardised one-step prediction residuals (innovations) of a correctly specified state-space model should be white noise. The Ljung and Box test on 24 lags gives p = < 0.001 for the full BSM and p = < 0.001 for the fixed slope version. Both models fail: there is clear autocorrelation left in the innovations (Figure 7). The irregular variance is estimated at or near zero, so all short-run variation has to be absorbed by the random walk level, and the model has no way to represent the short-lived, autocorrelated deviations that the ARIMA error in the regression model captures. By the Box and Jenkins rule, the BSM should not be used to forecast without being extended (for example with an AR(1) component), which the StructTS() function does not allow. Its forecasts are still evaluated below, but as a reference rather than a candidate.

# 6 Forecast evaluation

## 6.1 Forecasts on the test set

All models produce forecasts for July 2024 to June 2026 from the end of June 2024, with 80% and 95% prediction intervals. Two reference models are included: the seasonal naive benchmark and the SARIMA$(0,1,1)(0,1,1)_{12}$ model from Project 1 (Design B), refitted here with exactly the same specification so that the comparison uses identical code and data. For models fitted on the log scale, point forecasts are back-transformed to the mean (with the bias adjustment), as in Project 1.

```
# ETS, Project 1 SARIMA and seasonal naive (fable)
fit_ref <- train_B |>
  model(`SARIMA (Project 1)` = ARIMA(log(nights) ~ 0 + pdq(0, 1, 1) + PDQ(0, 1, 1)))
fit_sn  <- train |> model(`Seasonal naive` = SNAIVE(nights))

fc_fable <- bind_rows(
  forecast(fit_ets, h = 24) |> as_tibble(),
  forecast(fit_ref, h = 24) |> as_tibble(),
```

```
  forecast(fit_sn,  h = 24) |> as_tibble(),
  forecast(fit_reg |> select(Reg), new_data = test_reg) |> as_tibble(),
  forecast(fit_reg |> select(`Reg + ZAR`),
           new_data = test_reg |> mutate(log_zar = last(train$log_zar))) |>
    as_tibble() |> mutate(.model = "Reg + ZAR (ex ante)"),
  forecast(fit_reg |> select(`Reg + ZAR`), new_data = test_reg) |>
    as_tibble() |> mutate(.model = "Reg + ZAR (ex post)")
)

fc_fable_tbl <- fc_fable |>
  mutate(lo80 = quantile(nights, 0.10), hi80 = quantile(nights, 0.90),
         lo95 = quantile(nights, 0.025), hi95 = quantile(nights, 0.975)) |>
  select(.model, Month, mean = .mean, lo80, hi80, lo95, hi95)

# BSM forecasts (StructTS): normal on the log scale, back-transformed
bsm_fc <- function(fit, label) {
  p <- predict(fit, n.ahead = 24)
  m <- as.numeric(p$pred); s <- as.numeric(p$se)
  tibble(.model = label, Month = test$Month,
         mean = exp(m + s^2 / 2),
         lo80 = exp(m - qnorm(0.90) * s),  hi80 = exp(m + qnorm(0.90) * s),
         lo95 = exp(m - qnorm(0.975) * s), hi95 = exp(m + qnorm(0.975) * s))
}

fc_all <- bind_rows(
  fc_fable_tbl,
  bsm_fc(bsm_free,  "BSM"),
  bsm_fc(bsm_fixed, "BSM (fixed slope)")
) |>
  left_join(as_tibble(test) |> select(Month, actual = nights), by = "Month")

# MASE scale: in-sample seasonal naive MAE on the observed training data
mase_scale <- mean(abs(diff(train$nights, lag = 12)))

model_order <- c("Seasonal naive", "SARIMA (Project 1)", "ETS", "Reg",
                 "Reg + ZAR (ex ante)", "Reg + ZAR (ex post)",
                 "BSM", "BSM (fixed slope)")

results <- fc_all |>
  group_by(Model = .model) |>
  summarise(
    RMSE = sqrt(mean((actual - mean)^2)),
    MAE  = mean(abs(actual - mean)),
    MAPE = 100 * mean(abs(actual - mean) / actual),
    MASE = mean(abs(actual - mean)) / mase_scale,
    cover80 = mean(actual >= lo80 & actual <= hi80),
    cover95 = mean(actual >= lo95 & actual <= hi95),
    width95 = mean(hi95 - lo95),
    .groups = "drop"
  ) |>
```

```
  mutate(Model = factor(Model, levels = model_order)) |>
  arrange(Model)

diag_status <- tibble(
  Model = factor(model_order, levels = model_order),
  `Passes diagnostics` = c("Benchmark", "Yes (Project 1)",
                           NA, if_else(lbp("Reg") > 0.05, "Yes", "No"),
                           rep(if_else(lbp("Reg + ZAR") > 0.05, "Yes", "No"), 2),
                           if_else(lb_free > 0.05, "Yes", "No"),
                           if_else(lb_fixed > 0.05, "Yes", "No"))
)

ets_lb <- augment(fit_ets) |>
  as_tibble() |>
  summarise(p = Box.test(.innov, lag = 24, type = "Ljung-Box")$p.value) |>
  pull(p)
diag_status$`Passes diagnostics`[diag_status$Model == "ETS"] <-
  if_else(ets_lb > 0.05, "Yes", "No")

results <- results |> left_join(diag_status, by = "Model")

results |>
  mutate(cover80 = pct(cover80), cover95 = pct(cover95)) |>
  kable(digits = c(0, 1, 1, 2, 2, 0, 0, 0, 0),
        col.names = c("Model", "RMSE", "MAE", "MAPE (%)", "MASE",
                      "Cover 80%", "Cover 95%", "Mean 95% width", "Passes
                       ↪  diagnostics"),
        format.args = list(big.mark = ","))
```

Table 4: Test set accuracy, July 2024 to June 2026. RMSE, MAE and mean 95% interval width are in thousands of nights. MASE uses the in-sample seasonal naive error as its scale, the same for every model. The last column records whether the model passed the residual diagnostics (Ljung and Box, 24 lags, 5% level).

| Model | RMSE | MAE | MAPE (%) | MASE | Cover 80% | Cover 95% | Mean 95% width | Passes diagnostics |
|---|---|---|---|---|---|---|---|---|
| Seasonal naive | 132.6 | 112.2 | 5.81 | 0.32 | 100% | 100% | 2,898 | Benchmark |
| SARIMA (Project 1) | 101.6 | 81.4 | 4.25 | 0.23 | 92% | 100% | 545 | Yes (Project 1) |
| ETS | 130.0 | 112.1 | 5.90 | 0.32 | 100% | 100% | 2,292 | Yes |
| Reg | 98.8 | 85.7 | 4.51 | 0.25 | 100% | 100% | 1,060 | Yes |
| Reg + ZAR (ex ante) | 106.2 | 91.4 | 4.81 | 0.26 | 100% | 100% | 972 | Yes |
| Reg + ZAR (ex post) | 130.3 | 109.5 | 5.75 | 0.32 | 100% | 100% | 957 | Yes |
| BSM | 184.7 | 166.7 | 8.62 | 0.48 | 100% | 100% | 933 | No |
| BSM (fixed slope) | 102.9 | 86.8 | 4.49 | 0.25 | 100% | 100% | 988 | No |

```
show_models <- c("SARIMA (Project 1)", "ETS", "Reg", "BSM (fixed slope)")
```

```r
hist_df <- nights_sold |>
  as_tibble() |>
  filter(Month >= yearmonth("2022 Jan"))

fc_all |>
  filter(.model %in% show_models) |>
  mutate(.model = factor(.model, levels = show_models)) |>
  ggplot(aes(as.Date(Month))) +
  geom_ribbon(aes(ymin = lo95, ymax = hi95), fill = "steelblue", alpha = 0.25) +
  geom_line(aes(y = mean), colour = "steelblue4", linewidth = 0.6) +
  geom_line(data = hist_df, aes(as.Date(Month), nights), colour = "black", linewidth
  ↪  = 0.4) +
  facet_wrap(vars(.model), ncol = 2) +
  scale_y_continuous(labels = scales::label_comma()) +
  labs(x = NULL, y = "Nights sold (thousands)")
```

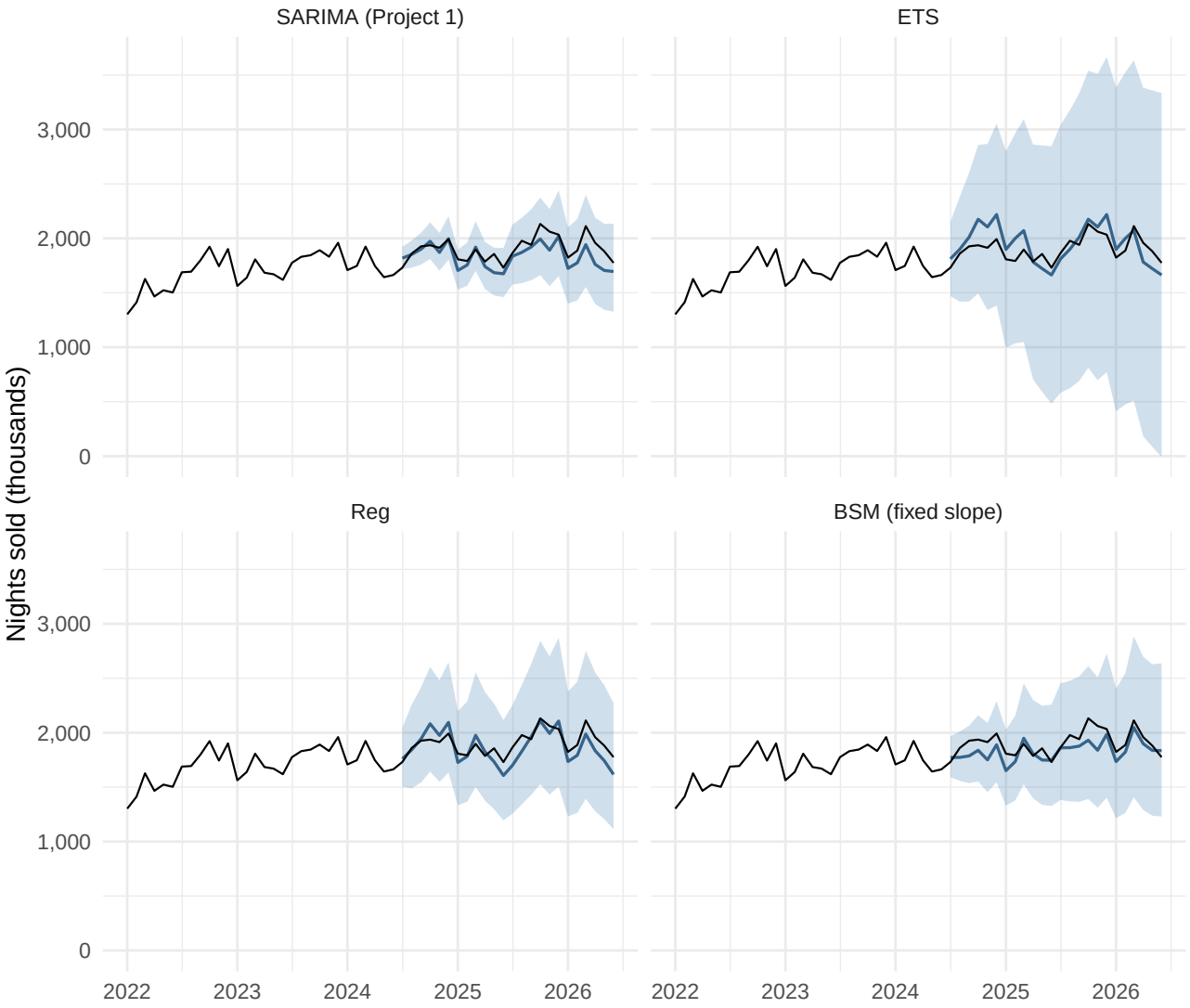



Figure 8: Test set forecasts and 95% prediction intervals for the main models. Black: observed values.

## 6.2 Results

**Overall ranking.** The lowest test RMSE is achieved by the **Reg** model (98.8 thousand nights). Among the models that passed the diagnostic step, the best is the **Reg** model. For reference, the Project 1 SARIMA has an RMSE of 101.6 and the seasonal naive benchmark 132.6.

**ETS.** ETS reaches an RMSE of 130.0, beating the seasonal naive benchmark even though it was fitted to data that include the COVID collapse. Its fast-moving level lets it recover from the shock without any special treatment, which a SARIMA with the COVID months included could not do in Project 1. Its 95% intervals are, however, wider than those of the regression model, because the COVID months still inflate its estimated error variance (mean width 2,292 against 1,060 thousand nights).

**Regression with intervention terms.** Modelling the shock explicitly gives an RMSE of 98.8 and a MAPE of 4.51%, better than the Project 1 SARIMA that simply deleted the COVID months. Its intervals cover 100% of the test months at the 95% level and 100% at the 80% level. The two approaches reach very similar forecast accuracy, which makes sense: by June 2024 the temporary part of the shock

had almost disappeared ($\hat{\delta}^{50} \approx$ 1.1e-06), so the main forecasting information in both models comes from the post-COVID level and the seasonal pattern. The advantage of the regression model is not mainly accuracy but *interpretation*: it estimates the size, shape and persistence of the shock (Figure 4) instead of discarding it.

**Exchange rate.** Adding the rand changes the RMSE to 106.2 ex ante and 130.3 ex post. Neither version improves on the model without the exchange rate. The gap between the ex ante and ex post results is a reminder that a regression model's forecasts are only as good as the forecasts of its explanatory variables.

**State-space models.** The full BSM has an RMSE of 184.7 and the fixed slope version 102.9. The difference comes from the slope: in the full model the slope adapts to the steep recovery in 2022 and 2023 and this growth is extrapolated forward, whereas with a fixed slope the long-run average growth is used. This shows that two models with almost identical likelihoods can give very different forecasts, because the likelihood is dominated by one-step-ahead fit while forecasts two years ahead depend on how the trend is extrapolated. Both BSMs failed the residual diagnostics, so even if their forecasts happen to be competitive on this test set, they are not candidates under the Box and Jenkins rule.

## 7 Comparison with Project 1

```
tibble(
  `Treatment of the COVID months` = c(
    "Kept, no special treatment (Project 1, Design A)",
    "Kept, level adapts quickly (ETS)",
    "Treated as missing, Kalman filter inside ARIMA (Project 1, Design B)",
    "Treated as missing, structural model (BSM, fixed slope)",
    "Modelled with intervention terms (regression with ARIMA errors)"
  ),
  Model = c("SARIMA (0,1,1)(0,1,1)", "ETS", "SARIMA (0,1,1)(0,1,1)",
            "BSM (fixed slope)", "Reg"),
  `Test RMSE` = c("see Project 1",
                  fmt(c(rg("ETS", "RMSE"), rg("SARIMA (Project 1)", "RMSE"),
                        rg("BSM (fixed slope)", "RMSE"), rg("Reg", "RMSE"))))
) |>
  kable()
```

Table 5: How the COVID-19 break was handled across Projects 1 and 3, with test set RMSE (thousands of nights). The Project 1 COVID-kept SARIMA result is taken from the Project 1 report.

| Treatment of the COVID months | Model | Test RMSE |
|---|---|---|
| Kept, no special treatment (Project 1, Design A) | SARIMA (0,1,1)(0,1,1) | see Project 1 |
| Kept, level adapts quickly (ETS) | ETS | 130.0 |
| Treated as missing, Kalman filter inside ARIMA (Project 1, Design B) | SARIMA (0,1,1)(0,1,1) | 101.6 |
| Treated as missing, structural model (BSM, fixed slope) | BSM (fixed slope) | 102.9 |
| Modelled with intervention terms (regression with ARIMA errors) | Reg | 98.8 |

Across the two projects, the same series has been forecast with five different treatments of the structural break (Table 5; the Design A result is reported in Project 1, where it was far worse than every other approach). The main lesson is that *any* sensible treatment of the break (letting the level adapt, deleting the months, or modelling them) does far better than ignoring it inside a model that assumes constant parameters. Between the sensible treatments, the differences in forecast accuracy are much smaller than the differences in what each model tells us: only the regression model quantifies the shock, only the BSM gives an estimate of the underlying level through the gap, and only ETS needed no decision about the COVID window at all.

# 8 Discussion

## 8.1 Key findings

1. **Deterministic seasonality.** The seasonal MA coefficient of -1 found in Project 1 is evidence of a fixed seasonal pattern. Replacing seasonal differencing with monthly dummies gave a regression model with white noise residuals (Ljung and Box p = 0.556).
2. **The shock can be modelled parsimoniously.** A decaying intervention with $\hat{\delta} = 0.76$ (half-life about 2.5 months), two pulses and a level shift capture the collapse and recovery with five parameters instead of 25 deleted months.
3. **Forecast accuracy.** The best model on the test set was Reg, and the best model that also passed the diagnostics was Reg. The regression model (RMSE 98.8) and the Project 1 SARIMA (RMSE 101.6) were close, because by 2024 the temporary part of the shock had died out.
4. **ETS is a robust baseline.** Without any special treatment of COVID, ETS reached an RMSE of 130.0, because its level adapts quickly.
5. **The exchange rate** had an estimated elasticity of 0.270 (p 0.175). Ex ante, it did not improve the forecasts.
6. **The BSM** interpolates the level through the COVID gap and its forecasts depend strongly on whether the slope is allowed to vary, but its innovations were not white noise, so it failed the Box and Jenkins diagnostic step.

## 8.2 Issues and limitations

- **Single test period.** All comparisons rest on one 24 month test set. Project 1 showed that rolling-origin cross-validation gives a more reliable ranking; with more time I would repeat it here, although refitting the BSM and repeating the $\delta$ profile at every origin is slow.
- **Choice of intervention shape.** The decay, pulses and step were chosen by looking at the data, so there is a risk of over-fitting the shape of the shock to this particular series. The AICc profile guards against some of this, but the pulse months were picked by eye.
- **Confounding with the exchange rate.** The rand depreciated sharply in March and April 2020, at the same time as the lockdown. Adding the exchange rate after the intervention terms avoids attributing the COVID collapse to the rand, but it also means the exchange rate effect is estimated mainly from the non-COVID months.
- **Same-month effect only.** The exchange rate enters without a lag. Foreign visitors book weeks or months ahead, so a lagged effect would be more realistic and would also make ex ante forecasting easier for short horizons.

- **BSM limitations in `StructTS()`.** The irregular variance was estimated at zero and the innovations were autocorrelated. A more flexible state-space model (for example in `dlm` or `KFAS`, with an AR(1) component or explanatory variables in the state equation) would be the natural next step.
- **Profile likelihood standard errors.** Treating $\hat{\delta}$ as known when computing the other standard errors understates their uncertainty.

### 8.3 What I would do differently

I would estimate $\delta$ jointly with the other parameters by maximising the likelihood directly, use rolling-origin cross-validation for the final ranking, try a lagged exchange rate, and build the state-space model in `dlm` so that an AR(1) component and the intervention terms could be added to it, combining the strengths of the regression and state-space approaches in one model.

## Reproducibility

All results are produced by the code in this document from `tourist_data.xlsx` (Stats SA P6410) and `usdzar_monthly_fred.csv` (FRED EXSFUS). The first render downloads the exchange rate file from FRED and saves it; later renders reuse the saved file.

```
R version 4.5.3 (2026-03-11 ucrt)
Platform: x86_64-w64-mingw32/x64
Running under: Windows 11 x64 (build 26200)

Matrix products: default
  LAPACK version 3.12.1

locale:
[1] LC_COLLATE=English_South Africa.utf8  LC_CTYPE=English_South Africa.utf8
[3] LC_MONETARY=English_South Africa.utf8 LC_NUMERIC=C
[5] LC_TIME=English_South Africa.utf8

time zone: Africa/Johannesburg
tzcode source: internal

attached base packages:
[1] stats     graphics  grDevices utils     datasets  methods   base

other attached packages:
 [1] knitr_1.51        readxl_1.5.0      fable_0.5.0       feasts_0.5.0
 [5] fabletools_0.8.0  ggtime_0.2.0      tsibbledata_0.4.1 tsibble_1.2.0
 [9] ggplot2_4.0.3     lubridate_1.9.5   tidyr_1.3.2       dplyr_1.2.0
[13] tibble_3.3.1      fpp3_1.0.3

loaded via a namespace (and not attached):
 [1] rappdirs_0.3.4       generics_0.1.4       anytime_0.3.13
 [4] digest_0.6.39        magrittr_2.0.4       evaluate_1.0.5
```

```
 [7] grid_4.5.3           timechange_0.4.0     RColorBrewer_1.1-3
[10] fastmap_1.2.0        cellranger_1.1.0     jsonlite_2.0.0
[13] purrr_1.2.2          scales_1.4.0         numDeriv_2016.8-1.1
[16] cli_3.6.5            rlang_1.1.7          crayon_1.5.3
[19] withr_3.0.3          yaml_2.3.12          otel_0.2.0
[22] tools_4.5.3          vctrs_0.7.1          R6_2.6.1
[25] lifecycle_1.0.5      pkgconfig_2.0.3      progressr_1.0.0
[28] pillar_1.11.1        gtable_0.3.6         glue_1.8.0
[31] Rcpp_1.1.2           xfun_0.56            tidyselect_1.2.1
[34] rstudioapi_0.19.0    farver_2.1.2         htmltools_0.5.9
[37] patchwork_1.3.2      rmarkdown_2.31       labeling_0.4.3
[40] compiler_4.5.3       S7_0.2.1             distributional_0.8.1
```